

Understanding class definitions 3

Exploring source code



- Produced Ms. Mairead Meagher,
 - by: Ms. Siobhán Roche.

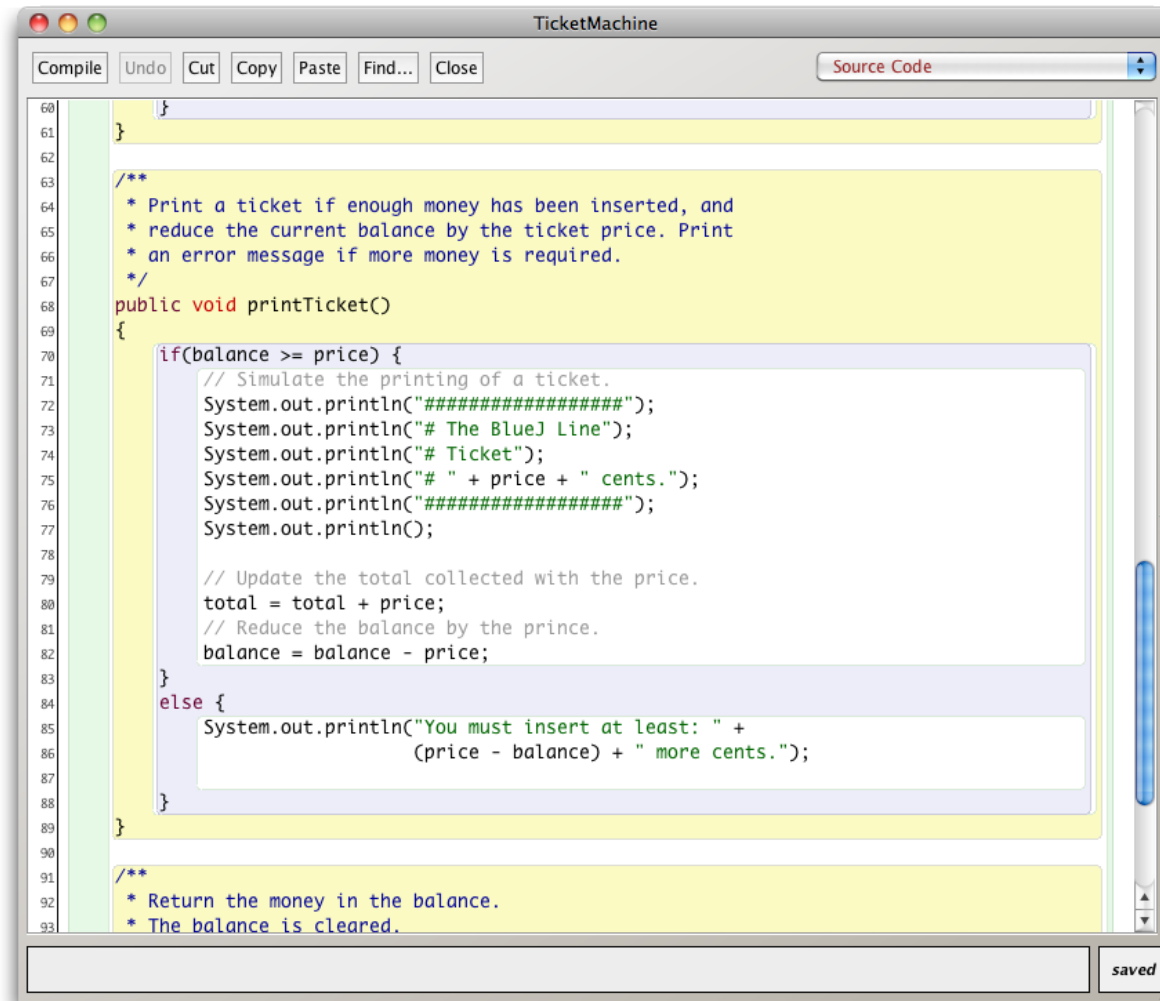
Upcoming

- Local variables.

Variables – a recap

- Fields are one sort of variable.
 - They store values through the life of an object.
 - They are accessible throughout the class.
- Parameters are another sort of variable:
 - They receive values from outside the method.
 - They help a method complete its task.
 - Each call to the method receives a fresh set of values.
 - Parameter values are short lived.

Scope highlighting



```
60 }
61 }
62
63 /**
64  * Print a ticket if enough money has been inserted, and
65  * reduce the current balance by the ticket price. Print
66  * an error message if more money is required.
67  */
68 public void printTicket()
69 {
70     if(balance >= price) {
71         // Simulate the printing of a ticket.
72         System.out.println("#####");
73         System.out.println("# The BlueJ Line");
74         System.out.println("# Ticket");
75         System.out.println("# " + price + " cents.");
76         System.out.println("#####");
77         System.out.println();
78
79         // Update the total collected with the price.
80         total = total + price;
81         // Reduce the balance by the price.
82         balance = balance - price;
83     }
84     else {
85         System.out.println("You must insert at least: " +
86             (price - balance) + " more cents.");
87     }
88 }
89
90
91 /**
92  * Return the money in the balance.
93  * The balance is cleared.
```

Scope and lifetime

- Each block defines a new scope.
 - Class, method and statement.
- Scopes may be nested:
 - statement block inside another block inside a method body inside a class body.
- Scope is static (textual).
- Lifetime is dynamic (runtime).

To think about

- How could we write a method to 'refund' an excess balance?
 - We need to return the value in balance.
 - We need to set the balance to zero.
- How can we do the first without losing the value in balance?

Unsuccessful attempt

```
public int refundBalance()  
{  
    // Return the amount left.  
    return balance;  
    // Clear the balance.  
    balance = 0;  
}
```

It looks logical, but the language does not allow it.

Local variables

- Methods can define their own, *local* variables:
 - Short lived, like parameters.
 - The method sets their values – unlike parameters, they do not receive external values.
 - Used for ‘temporary’ calculation and storage.
 - They exist only as long as the method is being executed.
 - They are only accessible from within the method.
 - They are defined within a particular *scope*.

Local variables

A local variable



No visibility
modifier

```
public int refundBalance()  
{  
    int amountToRefund;  
    amountToRefund = balance;  
    balance = 0;  
    return amountToRefund;  
}
```

Scope and lifetime

- The scope of a field is its whole class.
- The lifetime of a field is the lifetime of its containing object.
- The scope of a local variable is the block in which it is declared.
- The lifetime of a local variable is the time of execution of the block in which it is declared.

Review (1)

- Class bodies contain fields, constructors and methods.
- Fields store values that determine an object's state.
- Constructors initialise objects – particularly their fields.
- Methods implement the behavior of objects.

Review (2)

- Fields, parameters and local variables are all variables.
- Fields persist for the lifetime of an object.
- Local variables are used for short-lived temporary storage.
- Parameters are used to receive values into a constructor or method.

Review (3)

- Methods have a return type.
- **void** methods do not return anything.
- non-**void** methods always return a value.
- non-**void** methods must have a return statement.

Review (4)

- ‘Correct’ behavior often requires objects to make decisions.
- Objects can make decisions via conditional (if) statements.
- A true-or-false test allows one of two alternative courses of actions to be taken.

Questions?

